



RPS Project Report : Dataset Team

Vidyashilp University

MLOps and Model Deployment | DATA303 | Professor Venkata Babji

6th Semester, B.Tech Batch of 2023

Table Of Contents

1. Introduction.....	3
2. Data Collection Process.....	3
3. Structure of the Exported Dataset.....	4
4. Nature of the Annotation Problem.....	6
5. Comment Cleaning and Normalization Pipeline.....	7
5.1 Metadata Extraction.....	7
5.2 Basic Normalization.....	8
5.3 Advanced Normalization.....	8
5.4 Canonicalization.....	9
6. Comment-Level Results.....	10
7. Image-Level Label Assignment.....	11
8. Threshold Selection.....	12
9. Final Image-Level Dataset Status.....	12
10. Dataset Reduction Funnel.....	13
10.1 From 10,707 files to 4,683 media files.....	13
10.2 From 4,683 media files to 2,925 assessed image-level entries.....	14
10.3 From 2,925 assessed entries to 2,430 automatically labeled entries.....	14
10.4 From 2,430 labeled entries to 1,985 processed, exported images.....	14
10.5 Final interpretation.....	15
11. Audit and Quality-Control Outputs.....	15
12. Validation Through Simulated Testing.....	16
13. Image Preprocessing Stage.....	16
14. Model-Ready Packaging.....	18
15. Train-Only Horizontal Flip Augmentation.....	19
16. Challenges Encountered.....	20
16.1 Informal Annotation Medium.....	20
16.2 Annotation Quality Variability.....	20
16.3 Multi-Annotator Aggregation.....	20
16.4 Mixed Image Formats.....	21
16.5 Google Takeout Duplication and Naming Irregularities.....	21
16.6 Metadata-to-Image Mismatch During Export.....	21
16.7 Review Cases and Missing Usable Labels.....	21
16.8 Reproducibility Across Tools and Storage Systems.....	22
17. Strengths of the Data Stage.....	22
18. Conclusion.....	23
Contributions.....	24

1. Introduction

This report documents the full data-stage workflow undertaken for the Rock-Paper-Scissors image classification project. The objective of this stage was to build a reliable, well-documented, and reproducible dataset that could be handed over to the model development team for downstream machine learning tasks. The data stage, therefore, included data sourcing, annotation collection, export and structuring of the dataset, comment-based label cleaning, image-level label assignment, image preprocessing, augmentation planning and implementation, and preparation of model-ready outputs.

A central feature of this project is that the dataset was not gathered from a standard benchmark repository, nor was it labeled using a specialized annotation platform. Instead, the project used a collaborative collection and labeling workflow involving student contributors from the university. As a result, the data stage required significant effort not only in collection, but also in cleaning, normalization, aggregation, quality assurance, and export standardization before the dataset became suitable for machine learning use.

2. Data Collection Process

The raw data for this project were sourced from two cohorts of the university's BTech 2023 batch. Students were incentivized through marks for both image contribution and labeling contribution. This incentive structure was important because it encouraged participation at two levels: first, in expanding the visual dataset, and second, in providing labels for the collected images.

The collection process was organized through Google Photos. Initially, data was uploaded to a shared Google Photos link used for raw image collection:

[Collection album](#)

The collected content was then transferred to a second Google Photos link that was used exclusively for labeling through comments:

[Commenting album](#)

This two-album design separated raw data acquisition from annotation activity. Operationally, this was a practical decision. It reduced confusion between collection and labeling, and it allowed the second album to function as a lightweight annotation interface without requiring any custom-built tool.

After annotation was completed, the dataset was exported locally using Google Takeout. This export formed the basis of the repository's raw dataset source and allowed the project to move from a cloud-hosted collaborative environment into a reproducible local preprocessing workflow.

3. Structure of the Exported Dataset

The full exported dataset is stored within the Google Takeout directory used by this repository. The dataset contains both the images themselves and the metadata files generated through Google Takeout.

A crucial point is that the folder's total file count is much larger than the number of actual images, because Google Takeout exports both media and metadata.

The source export currently contains:

- 10,707 total files in the album folder
- 4,683 source media files used as candidate visual inputs
- 6,024 metadata JSON files

The media file breakdown is:

- 4,346 .jpg files
- 232 .heic files
- 59 .dng files
- 4 .png files
- 42 .mp4 files

The metadata side includes:

- 4,841 metadata files matched by the preprocessing pipeline
- additional JSON files associated with the Google Takeout structure

This distinction is important because the phrase “number of images in the dataset” can refer to different things:

- total files in the Takeout folder
- total media files
- total image-level entries assessed by the labeling pipeline
- total final labeled entries
- total final processed images exported for model handoff

These are not the same quantity.

This shows that the collected dataset is heterogeneous in file type and image origin. The presence of .heic and .dng files indicates that contributors uploaded images from multiple devices and camera pipelines rather than from a standardized acquisition setup. That heterogeneity increased the preprocessing burden later in the project.

An additional complexity of the Google Takeout export was the presence of duplicate-style naming patterns such as:

- image.jpg
- image(1).jpg
- image.jpg.supplemental-metadata.json
- image.jpg.supplemental-metadata(1).json

These required explicit handling in the preprocessing pipeline so that metadata records could be matched correctly to the appropriate image file.

Each metadata file contains image-level information as well as annotation content. Most importantly, the Google Photos comments were preserved in structured form and could be parsed automatically. This allowed comments to function as raw labels for the project.

4. Nature of the Annotation Problem

A major challenge in this project arose from the fact that labels were not collected using a controlled annotation interface with predefined classes. Instead, participants labeled the images by leaving natural-language comments in Google Photos. While this made participation easy and scalable, it also introduced substantial inconsistency.

The raw comments exhibited several types of noise:

- variation in capitalization, such as Rock, rock, and ROCK
- irregular spacing, such as extra spaces before or after the word
- singular versus plural variation, such as scissor and scissors
- synonym usage, such as stone for rock

- spelling mistakes, such as paper or scissors
- emojis and decorative symbols appended to labels
- irrelevant or unusable comments
- comments containing more than one class label, such as rock paper

This meant that the raw comments could not be used directly as training labels. A dedicated normalization and canonicalization pipeline was therefore required.

5. Comment Cleaning and Normalization Pipeline

The comment cleaning process is implemented in `process_rps_comments.py`. This script constitutes the main labeling and quality-control pipeline of the data stage.

5.1 Metadata Extraction

The script reads each metadata JSON file and extracts the following fields:

- image filename
- image metadata
- image creation time
- comment text
- comment owner
- comment timestamp

The updated version of the script also handles duplicate-style Google Takeout metadata names, such as `.supplemental-metadata(1).json`, and maps them to the corresponding copied image files, such as `image(1).jpg`, when present. This step preserves a full link between the source image and each human-provided comment.

5.2 Basic Normalization

The first layer of text cleaning applies basic normalization:

- Surrounding whitespace is removed
- text is converted to lowercase
- Repeated internal whitespace is collapsed

For example, a comment such as " Rock " becomes "rock".

This step addresses the most superficial inconsistencies without changing the basic semantic content of the comment.

5.3 Advanced Normalization

A stronger cleaning step is then performed:

- Unicode normalization
- punctuation removal
- decorative symbol and emoji removal
- preservation of alphabetic text and spaces
- whitespace cleanup

This step was designed to ensure that comments like "rock 🪨" or "Paper 📄" could still be mapped correctly to standard class labels.

5.4 Canonicalization

After normalization, comments are mapped into a fixed label space:

- rock
- paper
- scissors

- noise
- ambiguous

The canonicalization process uses multiple strategies:

- exact matching
- synonym mapping
- token-level phrase matching
- conservative typo correction using Levenshtein distance
- limited Hamming-distance and Soundex-style fallback support

Examples include:

- "stone" mapped to rock
- "papet" mapped to paper
- "sissors" mapped to scissors
- "rock paper" mapped to ambiguous
- "phone" mapped to noise

A key strength of this design is that it is conservative and explainable. It aims to standardize labels without over-interpreting uncertain input.

6. Comment-Level Results

The current normalization statistics, from `output/normalization_stats.csv`, are as follows:

- Total comments processed: 5,565
- Unique raw comment variants: 117

- Unique variants after basic normalization: 78
- Unique variants after advanced normalization: 61

Final canonical label counts are:

- rock: 1,530
- paper: 1,526
- scissors: 1,561
- noise: 947
- ambiguous: 1

These results show that a large portion of comments were successfully converted into usable labels, while a non-trivial amount of annotation noise remained in the full dataset. The larger full export naturally introduced much more label variability than the earlier working subset.

The method breakdown is also informative:

- whole-string match: 4,551 comments
- Levenshtein-based typo correction: 58 comments
- unresolved noise: 944 comments
- exact or synonym token match: 6 comments
- empty after cleaning: 3 comments
- Soundex-based fallback: 2 comments
- multiple-label token detection: 1 comment

This suggests that most contributors still used labels close to the target vocabulary, but that the full dataset contained considerably more annotation noise than the earlier small subset.

7. Image-Level Label Assignment

Since each image could have more than one comment, the project needed a policy for assigning one final label per image. The data stage therefore, included an image-level aggregation process rather than treating each comment independently.

For each image, the pipeline computes:

- The number of rock, paper, and scissors votes
- the number of noise comments
- The number of ambiguous comments
- the majority label
- the number of majority votes
- the majority ratio

The decision policy is:

- if an image has no valid class comments, it is sent to review
- if the majority ratio is above the threshold and no ambiguous comments are present, the majority class is assigned as the final label
- Otherwise, the image is sent to review

This policy gives the final dataset a clearer supervisory signal than raw comments alone.

8. Threshold Selection

Three thresholds were evaluated:

- 0.5
- 0.6
- 0.75

The threshold comparison outputs show:

- The threshold 0.5 produced 488 review cases
- A threshold of 0.6 produced 495 review cases
- A threshold 0.75 produced 500 review cases

The threshold of 0.6 was retained because it continues to provide a reasonable balance between automatic coverage and conservatism. Threshold 0.5 yields slightly fewer review cases, but 0.6 offers a somewhat stricter acceptance rule. Threshold 0.75 was again judged too conservative because it increases the manual review burden further.

This threshold choice remains an important methodological decision, as it balances confidence with dataset usability.

9. Final Image-Level Dataset Status

According to `output/image_label_summary.csv`, the final image-level label counts are:

- paper: 788
- rock: 792
- scissors: 850
- review: 495

This yields a total of 2,925 assessed image-level entries, of which 2,430 were automatically assigned a final class label. A total of 495 entries remained in the review queue.

The review queue is especially informative. At the chosen threshold, the review cases were distributed as follows:

- 485 due to no_valid_comments
- 9 due to low_majority_ratio
- 1 due to ambiguous_comments_present

This indicates that unresolved items were still driven primarily by unusable annotation content rather than major disagreement among annotators.

The majority ratio among accepted non-review images ranges from 0.6667 to 1.0, meaning that even the weakest accepted cases still had a two-thirds class majority.

10. Dataset Reduction Funnel

One of the most important interpretive points in this project is that the final handoff dataset is much smaller than the total number of files in the Takeout folder. This reduction is expected and occurs in several stages.

10.1 From 10,707 files to 4,683 media files

The top-level Takeout album folder contains 10,707 files, but that total includes both media and metadata JSON files. Only 4,683 of these are media files.

Therefore, the total file count should not be interpreted as the number of usable images.

10.2 From 4,683 media files to 2,925 assessed image-level entries

Not every media file in the folder becomes part of the labeling pipeline. An entry must have usable metadata and at least one extracted comment row to participate in image-level label assessment. As a result, only 2,925 image-level entries were actually assessed by the labeling stage.

Thus, raw file presence is not equivalent to annotation coverage.

10.3 From 2,925 assessed entries to 2,430 automatically labeled entries

Among assessed entries, not all satisfy the confidence and quality rules required for final label assignment. At threshold 0.6, 495 entries were sent to review, leaving 2,430 automatically labeled image-level entries.

This reduction reflects a label-quality filter, not a file-loss problem.

10.4 From 2,430 labeled entries to 1,985 processed, exported images

Finally, even after an entry receives a final label, it must still be possible to locate and successfully process the corresponding source image during the image-preprocessing stage. In the full export:

- 2,430 entries had final labels
- 1,985 were successfully exported as processed labeled images
- 445 were skipped during image export

This reduction reflects image-export feasibility, not annotation disagreement.

10.5 Final interpretation

The dataset, therefore, contracts through successive filters:

- raw Takeout files
- media-only files
- annotation-covered assessed entries
- confidently labeled entries
- successfully exportable processed images

This staged reduction explains why the final model-ready dataset is much smaller than the apparent size of the raw Takeout folder.

11. Audit and Quality-Control Outputs

One of the strongest aspects of the project's data stage is the presence of multiple audit outputs. These files make the preprocessing pipeline transparent and traceable.

Important outputs include:

- `comment_level_audit.csv`
- `raw_comment_counts.csv`
- `basic_normalized_counts.csv`
- `advanced_normalized_counts.csv`
- `canonical_label_counts.csv`
- `image_label_summary.csv`
- `review_queue.csv`
- `normalization_stats.csv`

Together, these outputs provide both traceability and evidence of data quality, which are important in any academic or engineering setting.

12. Validation Through Simulated Testing

To ensure that the normalization logic behaved as intended, the project also included a simulated test suite in `tests_simulated_comments.py`.

The suite covers:

- capitalization cases
- whitespace irregularities
- emojis
- common typos
- synonyms
- ambiguous phrases
- irrelevant noise

The current result is 18 out of 18 tests passed. This provides evidence that the normalization rules are not arbitrary but systematically verified.

13. Image Preprocessing Stage

Beyond label cleaning, the project also needed to standardize the images themselves. This was necessary because the collected images were not uniform in dimensions, aspect ratios, orientation metadata, or file format.

The integrated image preprocessing pipeline is implemented in `prepare_rps_ml_dataset.py`.

The preprocessing flow:

- reruns comment cleaning to ensure labels remain synchronized
- filters to the final labeled images
- Excludes review images from the training export by default
- Opens images with Pillow
- applies EXIF orientation correction
- converts all images to RGB
- resizes each image while preserving aspect ratio
- pads the result to a square canvas
- saves processed outputs grouped by class
- writes manifests and configuration files for reproducibility
- optionally creates train/validation/test splits
- optionally creates train-only horizontal-flip augmentation

Default preprocessing settings are:

- image size: 128×128
- pad color: black
- output format: JPEG
- JPEG quality: 95
- resample filter: Lanczos

This stage ensures that the model team receives a uniform set of images, even though the raw data originated in mixed formats such as JPG, HEIC, DNG, and PNG.

14. Model-Ready Packaging

The final output of the data stage is not merely a cleaned label table, but a model-ready dataset package. The integrated handoff workflow can generate:

- processed image folders by class
- a full manifest file
- a review manifest
- a skipped-images report
- a dataset configuration file
- optional train/validation/test splits
- optional zip archives for transfer
- optional train-only horizontal-flip augmentation for the training split

On the final full export:

- 1,985 labeled images were successfully exported into the model-ready package
- 445 entries were skipped during image export
- 495 review images were listed separately rather than included in the training data

The skipped-image reasons were:

- 423 missing_source_image
- 22 open_or_resize_failed:UnidentifiedImageError

This means that the metadata-based labeling stage and the final image-export stage operate at slightly different usable scales, which is an important practical observation for downstream ML work.

15. Train-Only Horizontal Flip Augmentation

A further extension of the preprocessing stage was the addition of optional train-only horizontal-flip augmentation. This was motivated by the concern that many images may have been captured using front-facing cameras, while the deployment interface could use a different camera orientation. In such a case, horizontal flipping may help improve robustness to orientation mismatch.

The augmentation strategy adopted in this project was deliberately conservative:

- only training images are augmented
- validation and test images remain unchanged
- review images are never augmented
- actual flipped image files are exported rather than leaving augmentation to the model team

The augmentation is recorded in the manifests through fields such as:

- `is_augmented`
- `augmentation_type`
- `source_base_image`

In the full augmented handoff build:

- 1,588 original training images were present
- 1,588 flipped training images were added

- the final training manifest therefore contained 3,176 training rows
- validation and test splits remained unaugmented

This design avoids evaluation leakage while still improving the diversity of training inputs.

16. Challenges Encountered

Several important challenges were encountered during the data stage.

16.1 Informal Annotation Medium

Using Google Photos comments as the annotation mechanism made participation convenient, but it also introduced uncontrolled natural-language variation. Unlike a proper annotation interface with fixed label buttons, comments permitted spelling variation, synonyms, emojis, irrelevant messages, and multi-label expressions. This significantly increased preprocessing complexity.

16.2 Annotation Quality Variability

Because participation was incentivized with marks, contribution scale improved, but annotation quality likely varied between participants. Some contributors may have labeled carefully, while others may have commented more casually or minimally. The pipeline therefore had to compensate for human inconsistency.

16.3 Multi-Annotator Aggregation

Images often had multiple comments, which is useful for consensus labeling, but also requires aggregation logic. The project had to define how to turn multiple free-text annotations into a single image-level label without introducing arbitrary decisions.

16.4 Mixed Image Formats

The dataset includes JPG, HEIC, DNG, PNG, and a small number of MP4 files. This heterogeneity is realistic for student-contributed data but introduces challenges in compatibility, decoding, preprocessing, and consistent export formatting.

16.5 Google Takeout Duplication and Naming Irregularities

The full Google Takeout export introduced duplicate-style filenames and metadata records such as (1) variants. These required explicit logic in the processing script to correctly match metadata to the correct copied image files. Without this handling, a substantial portion of the full dataset would have been undercounted or misread.

16.6 Metadata-to-Image Mismatch During Export

Although image-level label assignment succeeded for a large number of entries, some labeled metadata entries could not be exported into the final image package because the corresponding source images were missing or unreadable. This created a gap between the number of labeled entries and the number of fully exportable model-ready images.

16.7 Review Cases and Missing Usable Labels

A significant number of image entries could not be labeled automatically because they had no valid class comments, low-majority ratios, or ambiguous comment content. This required the explicit introduction of a review queue. While this is a strength from a quality-control perspective, it also reflects the practical challenge of incomplete or unusable annotation behavior.

16.8 Reproducibility Across Tools and Storage Systems

Because the pipeline spans Google Photos, Google Takeout, local file processing, and Git-based project management, reproducibility required explicit documentation and scripting. Without a structured pipeline, the process would have been difficult to rerun or audit.

17. Strengths of the Data Stage

Despite the challenges, the project demonstrates several strong data-engineering qualities:

- it uses real human-generated data rather than synthetic or prepackaged benchmark data
- it preserves provenance from raw collection through final preprocessing
- it explicitly models label uncertainty through review logic
- it uses explainable normalization rules
- it validates preprocessing behavior with tests
- it exports transparent audit artifacts
- it standardizes images for model training
- it supports reproducible re-generation with adjustable parameters
- it now supports train-only augmentation in a leakage-safe manner

These strengths make the dataset significantly more useful to the model team than a simple folder of images and comments would have been.

18. Conclusion

The data stage of this project successfully transformed a loosely structured, collaboratively collected image set into a curated and reproducible dataset for machine learning. The process began with data collection from two cohorts of the university's BTech 2023 batch, using Google Photos for both image acquisition and comment-based labeling. It then moved through Google Takeout export, metadata parsing, comment normalization, canonical label assignment, image-level vote aggregation, threshold-based quality control, image preprocessing, and training-safe augmentation support.

The final outputs demonstrate that the preprocessing pipeline was effective at full scale:

- 5,565 raw comments were processed
- 4,617 comments were mapped into valid class labels
- 2,925 image-level entries were assessed
- 2,430 image-level entries received final labels
- 495 image-level entries remained for manual review
- 1,985 labeled images were successfully exported into the final model-ready package
- train-only horizontal-flip augmentation doubled the training split from 1,588 to 3,176 rows

In addition, the project now supports model-ready packaging through standardized image preprocessing, manifests, skipped-image reporting, optional dataset splits, train-only augmentation, and reproducible export settings. As a result, the data team's work is not limited to raw collection; it culminates in a documented and transferable dataset artifact suitable for downstream modeling.

Contributions

This project was carried out by the Data Group as part of the MLOps and Model Deployment (DATA303) course.

Team Lead: Priyanka Bhatt (priyankabhatt0022@gmail.com)

Repository: <https://github.com/pbhatt0022/rps-experiment>

Team Members: Priyanka, Sachin, Tanmayee, Uzair, Ahmed, Raju, Jeromie, Shreerenu, Brunda, Madhuri, Preethi

Acknowledgement:

We sincerely thank our course instructor, Venkata Babji, for his valuable guidance, encouragement, and continuous support throughout this project. His support helped us carry out the work in a more structured, rigorous, and practical manner.